RUCHE · LEÇON

Leçon 0 — Le socle

Généré le 12/10/2026

Avant de construire les murs d'une maison, on coule les fondations, on amène l'eau et l'électricité, on installe la cabane de chantier. Personne ne les admire, mais tout repose dessus. La phase 0, c'est ça : il n'y a qu'un écran d'accueil et une route d'API, mais toute la suite viendra s'y brancher sans tout casser.

Cette leçon fait le tour du chantier, outil par outil, avec à chaque fois le vrai code du dépôt.

---

1. Un seul dépôt, plusieurs tiroirs : le monorepo

Ruche contient plusieurs programmes : l'API (le serveur), l'appli (téléphone et web), et des morceaux partagés entre les deux. Plutôt qu'un dépôt Git par programme, on range tout dans un seul dépôt, avec un tiroir par programme. C'est un **monorepo**.

Tiroir	Ce qu'il contient
`apps/api`	Le serveur, qui répond aux demandes de l'appli
`apps/app`	L'appli, la même pour iOS, Android et le web
`packages/i18n`	Les textes en français et en anglais
`packages/contracts`	La forme des données échangées entre l'appli et l'API
`infra`	La recette pour faire tourner le tout sur un serveur

> **Analogie**
>
> Un monorepo, c'est une grande boîte à outils avec des tiroirs étiquetés. Quand on change une pièce partagée (les traductions, par exemple), on voit tout de suite si un autre tiroir en souffre, parce que tout est vérifié ensemble.

Deux outils font tourner cette boîte :

- **pnpm** installe les bibliothèques et relie les tiroirs entre eux. Quand l'API écrit `import ... from '@ruche/i18n'`, pnpm sait que ça désigne le tiroir `packages/i18n`.
- **Turborepo** lance une même commande dans tous les tiroirs, dans le bon ordre, et retient ce qui n'a pas changé pour ne pas le refaire.

pnpm-workspace.yaml

```yaml
packages:
  - apps/*
  - packages/*
```

package.json (racine)

```json
"scripts": {
  "lint": "biome check .",
  "typecheck": "turbo run typecheck",
  "test": "turbo run test",
  "check": "pnpm lint && pnpm typecheck && pnpm test"
}
```

> **À retenir**
>
> `pnpm check` est la commande à lancer avant chaque commit : elle vérifie le style, les types et tous les tests de tous les tiroirs.

---

2. TypeScript : un correcteur d'orthographe pour le code

JavaScript laisse tout passer jusqu'au moment où le programme plante. **TypeScript** ajoute des *types* : on précise ce qu'une variable a le droit de contenir, et l'éditeur souligne les erreurs avant même de lancer le programme.

Exemple réel : la liste des langues gérées, et un type qui en découle automatiquement.

packages/i18n/src/index.ts

```ts
// `as const` fige le tableau : TypeScript sait qu'il contient exactement 'fr' puis
'en',
// et pas « n'importe quelle chaîne de caractères ».
export const supportedLanguages = ['fr', 'en'] as const

// Type dérivé du tableau ci-dessus : Language vaut 'fr' | 'en'.
export type Language = (typeof supportedLanguages)[number]
```

Si quelqu'un écrit plus tard `const langue: Language = 'de'`, l'éditeur le refuse : l'allemand n'est pas dans la liste. Et le jour où l'on ajoutera `'es'` au tableau, le type suivra tout seul.

Le projet active le **mode strict**, la version la plus exigeante du correcteur :

tsconfig.base.json (extrait)

```
// Mode strict : TypeScript refuse les zones floues (valeur peut-être absente, type implicite…).
"strict": true,
// tableau[i] peut être `undefined` si i dépasse la taille : TypeScript oblige à le prévoir.
"noUncheckedIndexedAccess": true,
```

> **Analogie**
>
> Le mode strict, c'est le professeur qui refuse « à peu près juste ». C'est agaçant au début, puis on réalise que la plupart des bugs naissent justement des « à peu près ».

---

3. Les tests d'abord

Un test est un petit programme qui vérifie qu'un autre programme fait ce qu'on attend. On les écrit **avant** le code : d'abord on décrit le résultat voulu, on lance le test, il échoue (normal, le code n'existe pas encore), puis on écrit le code jusqu'à ce qu'il passe.

> **Analogie**
>
> C'est comme écrire la grille de correction avant de rédiger la copie. On sait exactement ce qu'on vise, et on sait quand on a fini.

Exemple : la fonction qui choisit la langue d'un visiteur. Voici une partie de ses tests, écrits avant elle.

packages/i18n/src/i18n.test.ts

```
describe('resolveLanguage', () => {
  it('matches a regional variant to its base language', () => {
    expect(resolveLanguage(['en-US'])).toBe('en')
    expect(resolveLanguage(['fr-CA'])).toBe('fr')
  })

  it('keeps the first supported language in order of preference', () => {
    expect(resolveLanguage(['de-DE', 'en-GB', 'fr'])).toBe('en')
  })

  it('falls back to French when no preference is supported', () => {
    expect(resolveLanguage(['de', 'es-ES'])).toBe('fr')
  })
})
```

Trois mots suffisent pour lire un test : `describe` regroupe, `it` décrit un comportement en une phrase, `expect(...).toBe(...)` vérifie. Puis le code qui fait passer ces tests :

packages/i18n/src/index.ts

```ts
export const resolveLanguage = (preferences: readonly string[]): Language => {
  for (const preference of preferences) {
    // "EN-us " → "en-us" → ["en", "us"] → "en"
    const base = preference.trim().toLowerCase().split('-')[0] ?? ''
    if (isLanguage(base)) return base
  }
  return defaultLanguage
}
```

> **À retenir**
>
> Les noms des tests sont des phrases (« falls back to French when… »). Quand un test échoue, son nom dit tout de suite quelle règle est cassée.

---

4. Deux langues dès le premier jour

Aucun texte n'est écrit en dur dans les écrans. Chaque texte a une **clé** (`home.title`) et une valeur par langue. Le français sert de référence :

packages/i18n/src/locales/fr.ts (extrait)

```ts
export const fr = {
  home: {
    title: 'Bienvenue sur {{appName}}',
    comingSoon: 'Le fil arrive bientôt.',
  },
  // …
} as const
```

L'anglais doit avoir exactement la même forme. Pour le garantir, on fabrique un type à partir du français et on l'impose à l'anglais :

packages/i18n/src/locales/en.ts (extrait)

```ts
// Le type `Translation` (calqué sur le français) oblige à fournir exactement
// les mêmes clés : en oublier une ou en ajouter une de trop empêche la compilation.
export const en: Translation = {
  home: {
    title: 'Welcome to {{appName}}',
    comingSoon: 'The feed is coming soon.',
  },
  // …
}
```

Double sécurité : en plus du type, un test vérifie que les deux langues ont les mêmes clés, aucun texte vide, et les mêmes paramètres `{{...}}`.

> **Piège évité**
>
> Sans ce test, on pourrait traduire `'Bienvenue sur {{appName}}'` par `'Welcome!'`. L'écran anglais perdrait le nom de l'appli sans que personne ne s'en rende compte.

---

5. Zod : le vigile à l'entrée

TypeScript vérifie le code pendant qu'on l'écrit. Mais une fois le programme lancé, les données qui arrivent par Internet peuvent avoir n'importe quelle forme. **Zod** décrit la forme attendue et la contrôle à l'exécution.

packages/contracts/src/health.ts

```ts
export const HealthResponse = z
  .object({
    status: z.literal('ok'), // exactement la chaîne 'ok', rien d'autre
    version: z.string(),
  })

// z.infer fabrique le type TypeScript à partir du schéma : { status: 'ok'; version: string }.
export type HealthResponse = z.infer<typeof HealthResponse>
```

> **Analogie**
>
> Zod est le vigile qui vérifie chaque badge à l'entrée. TypeScript est le plan de l'immeuble : utile, mais il n'empêche personne d'entrer avec un faux badge.

On écrit la forme **une seule fois**, dans `packages/contracts`, et on obtient les deux : le contrôle à l'exécution et le type pour l'éditeur. L'API et l'appli lisent ce même fichier : elles ne peuvent pas se désaccorder.

---

6. L'API : un standard téléphonique

L'API reçoit des **requêtes** (« donne-moi l'état du serveur ») et renvoie des **réponses** (en JSON, un format de texte structuré). On utilise **Hono**, une bibliothèque légère pour écrire des serveurs.

> **Analogie**
>
> L'API est un standard téléphonique. Chaque appel arrive sur une **route** (un numéro de poste : `/health`). Avant d'être transféré, l'appel passe par l'accueil, qui note la langue de l'appelant : c'est un **middleware**.

Le middleware de langue, qui s'exécute pour chaque requête :

apps/api/src/app.ts

```ts
app.use(async (c, next) => {
  const language = resolveLanguage(parseAcceptLanguage(c.req.header('accept-language')))
  c.set('language', language)
  await next() // on laisse la route s'exécuter…
  c.header('Content-Language', language) // …puis on complète la réponse
})
```

La route `/health` : on la décrit avec sa réponse attendue, puis on dit quoi répondre.

apps/api/src/app.ts

```ts
const healthRoute = createRoute({
  method: 'get',
  path: '/health',
  responses: {
    200: {
      description: 'The API is up.',
      content: { 'application/json': { schema: HealthResponse } },
    },
  },
})

app.openapi(healthRoute, (c) => c.json({ status: 'ok' as const, version }, 200))
```

Parce que chaque route décrit sa réponse avec un schéma Zod, Hono génère tout seul la documentation de l'API à l'adresse `/openapi.json`.

Le chemin d'une requête, de bout en bout

Un visiteur anglophone tape une adresse qui n'existe pas. Que se passe-t-il ?

1. Son navigateur envoie la requête avec l'en-tête `Accept-Language: en-US,en;q=0.9,fr;q=0.5`.
2. Le middleware `secureHeaders` prépare les en-têtes de sécurité.
3. Le middleware de langue lit l'en-tête et retient `en`.
4. Aucune route ne correspond : Hono appelle `notFound`.
5. `notFound` fabrique la réponse avec le message anglais :

apps/api/src/app.ts

```
app.notFound((c) => {
  const language = languageOf(c)
  c.header('Content-Language', language)
  return c.json(errorBody(language, 'not_found'), 404)
})
```

1. Le visiteur reçoit `404` et `{"error":{"code":"not_found","message":"This page does not exist."}}`.

Le test qui garantit ce comportement :

apps/api/src/app.test.ts

```
it('answer in English when the client prefers it', async () => {
  const res = await app.request('/does-not-exist', {
    headers: { 'Accept-Language': 'en-US,en;q=0.9,fr;q=0.5' },
  })
  const body = ErrorResponse.parse(await res.json())

  expect(res.headers.get('content-language')).toBe('en')
  expect(body.error.message).toBe('This page does not exist.')
})
```

À retenir

`app.request(...)` envoie une vraie requête à l'API, mais en mémoire, sans réseau. Les tests sont ainsi rapides et ne dépendent de rien d'extérieur.

---

7. Une configuration vérifiée au démarrage

Le serveur lit des réglages dans son environnement (le port, le mode production…). Ils arrivent sous forme de texte, et peuvent être absents ou faux. On les passe aussi au vigile Zod :

apps/api/src/env.ts

```
const EnvSchema = z.object({
  // z.coerce.number() convertit le texte "8080" en nombre 8080, puis vérifie qu'il est valide.
  PORT: z.coerce.number().int().min(1).max(65535).default(3000),
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
})
```

> **Piège évité**
>
> Avec `PORT=abc`, un serveur non vérifié démarrerait de travers et planterait plus tard, loin de la vraie cause. Ici, il refuse de démarrer et dit pourquoi.

---

8. L'appli : un écran est une fonction

L'appli utilise **React Native** via **Expo** : on écrit un seul code, qui devient une vraie appli iOS, une vraie appli Android et un site web.

En React, un écran est une **fonction** qui renvoie ce qu'il faut afficher, avec une syntaxe proche du HTML (le JSX). Les **hooks**, ces fonctions qui commencent par `use…`, branchent l'écran sur une donnée qui change : quand la donnée change, l'écran se redessine tout seul.

apps/app/src/screens/HomeScreen.tsx (extrait simplifié)

```tsx
export function HomeScreen() {
  // useTranslation : donne t() pour traduire, et redessine l'écran quand la langue change.
  const { t, i18n } = useTranslation()

  return (
    <View style={styles.screen}>
      <Text accessibilityRole="header" style={styles.title}>
        {t('home.title', { appName: t('common.appName') })}
      </Text>
      {supportedLanguages.map((language) => (
        <Pressable
          key={language}
          accessibilityRole="button"
          accessibilityState={{ selected: language === i18n.language }}
          onPress={() => i18n.changeLanguage(language)}
        >
          <Text>{t(`common.languages.${language}`)}</Text>
        </Pressable>
      ))}
    </View>
  )
}
```

> **Analogie**
>
> Un hook, c'est un abonnement. `useTranslation()` abonne l'écran à la langue : à chaque changement, il reçoit la nouvelle édition et se met à jour, sans qu'on ait rien à faire.

Le test de l'écran agit comme un utilisateur : il cherche le bouton par son nom visible, appuie dessus, et vérifie que le texte a changé.

apps/app/src/screens/HomeScreen.test.tsx

```
it('lets the visitor switch language', async () => {
  await renderHome('fr')

  await fireEvent.press(screen.getByRole('button', { name: 'English' }))

  expect(screen.getByText('Welcome to Ruche')).toBeTruthy()
})
```

---

9. Docker : des conteneurs, comme au port

Sur le serveur, chaque programme tourne dans un **conteneur** : une boîte fermée qui contient le programme et tout ce dont il a besoin. Docker Compose décrit l'ensemble des boîtes et comment elles communiquent.

> **Analogie**
>
> Comme les conteneurs maritimes : peu importe le navire (ton ordinateur, le VPS), la boîte s'embarque et fonctionne pareil partout.

Le point de sécurité le plus important de ce fichier : la base de données n'est joignable que par l'API, jamais depuis Internet.

infra/docker-compose.yml (extrait)

```
networks:
  # Réseau interne : la base n'est joignable que par l'API, jamais depuis Internet.
  backend:
    internal: true
  web:
```

Caddy est la porte d'entrée : il obtient tout seul les certificats HTTPS (le cadenas du navigateur) et transmet les requêtes à l'API.

---

10. La CI : un contrôle technique automatique

À chaque envoi de code sur GitHub, des machines de GitHub refont toutes les vérifications : style, types, tests, construction de l'appli et de l'image Docker, recherche de secrets oubliés, et failles connues dans les bibliothèques. C'est l'**intégration continue** (CI).

.github/workflows/ci.yml (extrait)

```
      - run: pnpm install --frozen-lockfile # échoue si pnpm-lock.yaml n'est pas à jour
      - run: pnpm lint
      - run: pnpm typecheck
      - run: pnpm test
      - run: pnpm build
```

À retenir

Une pull request n'est fusionnée que si la CI est verte. Même si Claude oublie de lancer les tests, la CI, elle, n'oublie jamais.

Les notions de cette leçon

Notion	En une phrase	Où la voir
Monorepo	Un dépôt, un tiroir par programme	`pnpm-workspace.yaml`
Type	Ce qu'une variable a le droit de contenir	`packages/i18n/src/index.ts`
Test	Un programme qui vérifie un comportement	`*.test.ts`
Schéma Zod	La forme d'une donnée, vérifiée à l'exécution	`packages/contracts`
Route	Une adresse de l'API et sa réponse	`apps/api/src/app.ts`
Middleware	Un passage obligé avant et après chaque route	`apps/api/src/app.ts`
Composant	Un écran (ou un morceau) écrit comme une fonction	`HomeScreen.tsx`
Hook	Un abonnement de l'écran à une donnée qui change	`useTranslation()`
Conteneur	Une boîte qui embarque un programme et ses besoins	`infra/docker-compose.yml`
CI	Les vérifications refaites à chaque envoi	`.github/workflows/ci.yml`

Exercice

But : voir de tes yeux le filet de sécurité des traductions.

1. Dans `packages/i18n/src/locales/fr.ts`, ajoute dans `home` une ligne `subtitle: 'Mon premier texte',`.

2. Lance `pnpm check`. Observe l'erreur de type : l'anglais ne respecte plus la forme du français.
3. Ajoute la même clé en anglais avec un texte vide (`subtitle: '',`) et relance. Cette fois, c'est le test « has no empty text » qui échoue.
4. Mets un vrai texte anglais, relance : tout redevient vert. Puis annule tes changements avec `git checkout packages/i18n`.

Bonus : lance l'API avec `PORT=abc pnpm --filter @ruche/api dev` et lis le message d'erreur.